Practice · Recursion

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What recursion is

What is a **recursive** function?

- A function that has a loop inside it.
- **Right answer:** A function that calls itself.
- A function that is called more than once from `main`.
- A function with no parameters.

Feedback: A recursive function solves a big problem by calling itself on a smaller one of the same kind: count down from 3 = say 3, then count down from 2. Class 8, slide 31.

(mc) L1 · The base case

```
void countdown(int n) {
    if (n == 0) {
        printf("Go!\n");
        return;
    }
    printf("%d\n", n);
    countdown(n - 1);
}
```

Which part is the **base case**?

- **Right answer:** `if (n == 0) { printf("Go!\n"); return; }`
- `printf("%d\n", n);`
- `countdown(n - 1);`
- `void countdown(int n)`

Feedback: The base case is the small case the function answers directly, with no more calls. Here: at 0, say `Go!` and stop. Without it the calls never stop. Class 8, slide 33.

(mc) L1 · The step closer

In `countdown(n - 1);`, why is it `n - 1` and not `n`?

- `n - 1` is faster to compute.
- It makes no difference.
- The compiler does not allow a call with `n`.
- **Right answer:** Each call must get closer to the base case. `countdown(n)` would call itself with the same number for ever.

Feedback: Every recursive function has two parts: a base case, and a step closer to it. `n - 1` moves 3 to 2 to 1 to 0, where the base case stops it. Class 8, slides 34 and 37.

(mc) L2 · countdown(2)

```
void go(int n) {
```

```
    if (n == 0) {
        printf("Go!\n");
        return;
    }
    printf("%d ", n);
    go(n - 1);
}
```

What does `go(2)` print?

- `Go! 2 1`
- **Right answer:** `2 1 Go!`
- `2 Go!`
- `2 1 0 Go!`

Feedback: `go(2)` prints 2 and calls `go(1)`. `go(1)` prints 1 and calls `go(0)`. `go(0)` is the base case: it prints `Go!` and returns. 0 is never printed, because the base case prints `Go!` instead. Class 8, slides 35 and 36.

(mc) L2 · How many calls

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

`main` calls `factorial(4)`. How many times does `factorial` run in all, counting that first call?

- 1
- 3
- **Right answer:** 4
- 24

Feedback: `factorial(4)` calls `factorial(3)`, which calls `factorial(2)`, which calls `factorial(1)`. That one is the base case and returns 1. Four calls down, then four returns back up: 1, 2, 6, 24. 24 is the answer it returns, not the number of calls. Class 8, slides 41 to 46.

(mc) L2 · power

```
int power(int base, int exp) {
    if (exp == 0) {
        return 1;
    }
    return base * power(base, exp - 1);
}
```

What does `power(2, 5)` return?

- `10`
- `25`
- `16`

- **Right answer:** `32`

Feedback: Each call takes 1 from `exp` and multiplies by `base`: 2 × 2 × 2 × 2 × 2 × 1 = 32. The base case, `exp == 0`, gives 1, because anything to the power 0 is 1. Class 8, slide 53.

(fib) L3 · The base case of sum_to

`sum_to(n)` adds 1 + 2 + ... + n. Complete the base case. Type only the number.

```
int sum_to(int n) {
    if (n == 0) {
        return [0];
    }
    return n + sum_to(n - 1);
}
```

Feedback: The sum of nothing is 0, so `sum_to(0)` returns 0. Then `sum_to(1)` is 1 + 0 = 1, `sum_to(2)` is 2 + 1 = 3, and `sum_to(4)` is 10. If the base case returned 1, every answer would be one too big. Class 8, slides 51 and 52.

(fib) L3 · The recursive call

Complete the call so that `countdown(3)` prints 3, 2, 1 and then `Go!`. Type only what goes inside the brackets.

```
void countdown(int n) {
    if (n == 0) {
        printf("Go!\n");
        return;
    }
    printf("%d\n", n);
    countdown([n - 1|n-1]);
}
```

Feedback: `n - 1` is the step closer: 3, then 2, then 1, then 0, where the base case stops. With `n` the function would call itself with 3 for ever; with `n + 1` it would move away from 0. Class 8, slides 32 and 34.

(fib) L3 · Trace sum_to(3)

```
int sum_to(int n) {
    if (n == 0) {
        return 0;
    }
    return n + sum_to(n - 1);
}
```

`sum_to(3)` returns [6].

Feedback: Down: `sum_to(3)` needs `sum_to(2)`, which needs `sum_to(1)`, which needs `sum_to(0)`, which returns 0. Back up: 1 + 0 = 1, 2 + 1 = 3, 3 + 3 = 6. Class 8, slides 51 and 52.